



GraphRAG & Knowledge Graphs

AICB-P2T3 · Ngày 19 · Chương 4 — Agent Nâng Cao

Giảng viên: Ngô Thanh Tùng

VinUniversity · Phase 2 · Track 3 · Tuần 4

"Khi user hỏi về mối quan hệ giữa 5 entities — flat RAG trả lời sai, GraphRAG trả lời đúng — tại sao?"



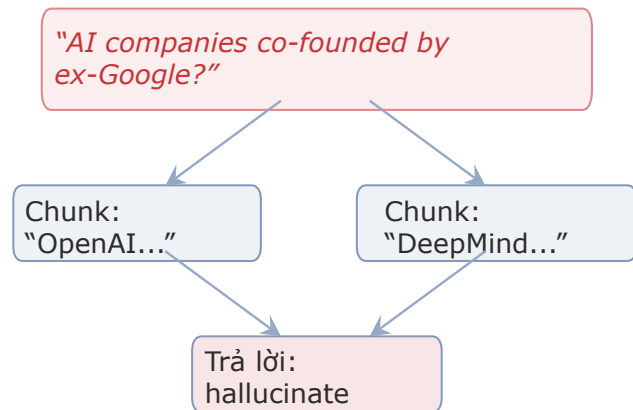
Giữ câu hỏi này trong đầu suốt buổi học hôm nay

1. Vấn đề của RAG: Khi Vector Search thất bại
2. Nền tảng về Knowledge Graph (KG)
3. Pipeline GraphRAG Tiêu chuẩn
4. Kiến trúc SOTA (Microsoft GraphRAG, LightRAG)
5. Chiến lược Doanh nghiệp & ROI (Tỷ suất hoàn vốn)
6. Thực hành (Lab): Xây dựng GraphRAG Agent

Khi nào Flat RAG thất bại?

Giới hạn của vector search khi cần suy luận quan hệ

1



Thiếu link giữa entities!

3 loại query Flat RAG struggle:

1. **Multi-hop relational:** “A liên kết B qua C”
2. **Global thematic:** “Tổng quan chủ đề X trong corpus”
3. **Cross-document:** “So sánh policy A với B”

Lưu ý: Vector RAG tìm câu **giống nhau**. Nhưng quan hệ giữa entities nằm ở **cấu trúc**, không phải ở embedding similarity.

"Những startup AI nào được đồng sáng lập bởi các cựu nhân viên Google, những người từng nghiên cứu về kiến trúc Transformer?"

Hệ thống RAG tiêu chuẩn sẽ xử lý câu hỏi này như thế nào?

Đòi hỏi liên kết 3 luồng thông tin:

❶ Startup AI & Nhà sáng lập

❷ Lịch sử làm việc tại Google

❸ Lịch sử dự án Transformer

→ Trải dài qua hàng chục tài liệu khác nhau.

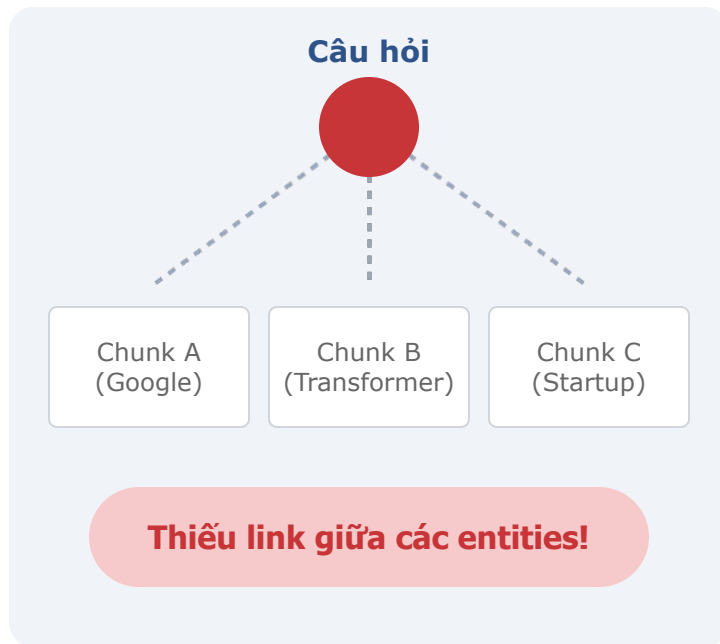
Kết quả: Sinh ra ảo giác (Hallucination)

Hoặc trả lời "Tôi không có đủ thông tin"

Tại sao?

Flat RAG truy xuất các đoạn văn bản (chunks) có sự tương đồng về ngữ nghĩa.

Nó có thể lấy ra một đoạn về Google, một đoạn về Transformer, và một đoạn về startup AI, nhưng thiếu đi mối liên kết giữa chúng.



Vector RAG (Flat RAG)

Tìm kiếm sự tương đồng về mặt ngữ nghĩa (Semantic similarity). Nó tìm các từ và khái niệm giống nhau trong các đoạn văn bản riêng lẻ.

Hạn chế

-  Không thể tự động duyệt qua các mối quan hệ. Giống như việc bạn tìm thấy 3 trang bách khoa toàn thư nhưng không đọc phần tham chiếu chéo.
-  Khi mối quan hệ trải dài qua nhiều tài liệu, Vector RAG sẽ gặp khó khăn.



GraphRAG

Hiểu được các kết nối mang tính cấu trúc. Nó tìm kiếm trên một "mạng lưới" các mối quan hệ thay vì chỉ các đoạn văn bản.



Sự dịch chuyển mô hình

Thay vì tìm "Văn bản giống nhau", chúng ta duyệt từ Node A → Node B → Node C.



Kết quả

Suy luận đa bước (multi-hop) chính xác và khả năng tóm tắt toàn cục toàn bộ tập dữ liệu.

Vector RAG vs GraphRAG

Vector RAG = tìm câu giống nhau trong sách — như search Google.

GraphRAG = hiểu mối quan hệ giữa các khái niệm — như đọc hiểu mind map.

Analogy: Bạn hỏi “Ai là bạn chung của An và Bình?”

Vector search tìm trang nói về An, trang nói về Bình — nhưng không link được. Graph search: An → bạn → Cường ← bạn ← Bình. Trả lời ngay:

Cường.

+40%

Comprehensiveness
vs Flat RAG

2–3×

Multi-hop
accuracy gain

OK

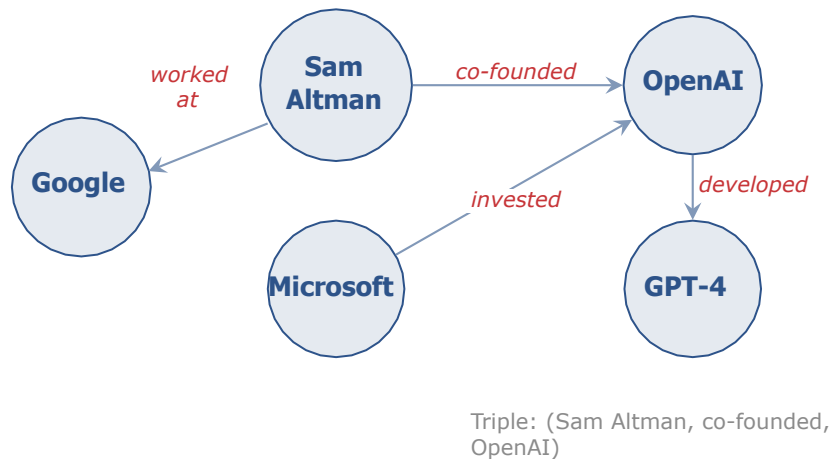
Flat RAG vẫn
tốt cho factoid

Benchmark trên community questions: GraphRAG +40% comprehensiveness. Nhưng single-doc factoid, Flat RAG nhanh hơn và rẻ hơn. **Không phải lúc nào cũng cần graph.**

Knowledge Graph Fundamentals

Nodes, Edges, Triples — nền tảng của graph-based retrieval

2



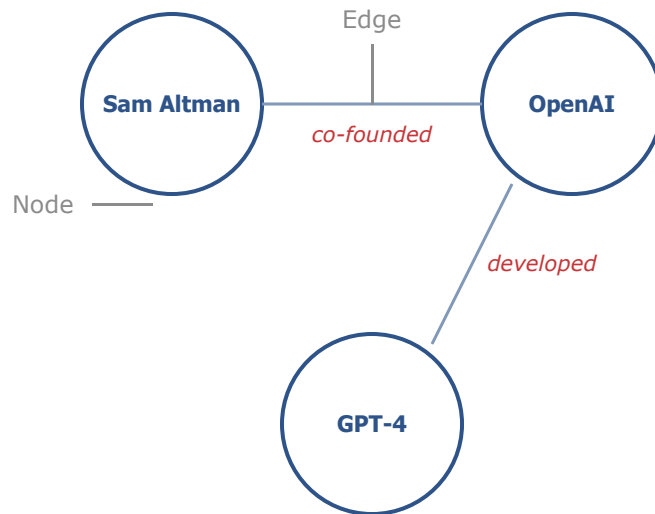
Knowledge Graph — Directed labeled graph: **Entity** (node) + **Relation** (edge) + **Triple** (subject-predicate-object)

- KG cho RAG: extract entities + relations, lưu graph DB
- Retrieval = **graph traversal** thay vì chunk search
- Microsoft GraphRAG (2024): pipeline open-source

Nodes (Đỉnh/Thực thể): Các thực thể trong dữ liệu

Edges (Cạnh/Mối quan hệ): Kết nối giữa các nodes

Properties (Thuộc tính): Metadata gắn với nodes hoặc edges



- Node Property: age=38
- Edge Property: year=2015

Knowledge Graphs được xây dựng từ các **Triples** (Bộ ba). Đây là nguyên tử cơ bản để cấu trúc hóa kiến thức nhân loại.

 **Cấu trúc:** (Chủ thể) → [Vị ngữ / Mối quan hệ] → (Tân ngữ)

Ví dụ 1:

(Sam Altman) → [CEO_OF] → (OpenAI)

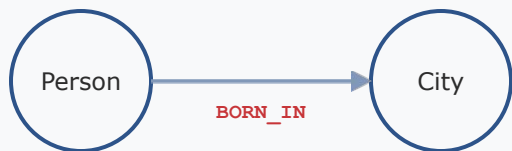
Ví dụ 2:

(OpenAI) → [DEVELOPED] → (GPT-4)

 Hàng triệu **Triples** này kết nối lại tạo thành một Knowledge Graph khổng lồ.

Đồ thị có hướng

Các cạnh có hướng cụ thể (đường một chiều).



Ví dụ: Chiều ngược lại không đúng.

Đồ thị vô hướng

Mỗi quan hệ hai chiều.



i Lưu ý cho GraphRAG: KG hầu như luôn là **Đồ thị có hướng và có nhãn** (Directed Labeled Graphs).

Property Graphs

 **Neo4j, FalkorDB:** Nodes và edges có thể chứa nhiều siêu dữ liệu.

 Hiệu năng cao cho truy xuất và ứng dụng doanh nghiệp.

 Tiêu chuẩn công nghiệp cho GraphRAG.

RDF

 **Resource Description Framework:** Tiêu chuẩn học thuật cho dữ liệu liên kết mở (Wikidata).

 Khó duy trì cho các hệ thống Generative AI thời gian thực.

🔍 Cách chúng ta truy vấn cơ sở dữ liệu đồ thị.

MATCH

Tìm kiếm mẫu
(pattern).

WHERE

Lọc dữ liệu.

RETURN

Trả về kết quả.

```
MATCH (p:Person) -[:CO_FOUNDED]->(c:Company)
WHERE c.name = 'OpenAI'
RETURN p.name
```



Đồ thị không tự sinh ra. Chúng ta phải trích xuất nodes và edges từ các văn bản thô, lộn xộn.



Đây là phần **khó nhất và tốn kém nhất** của GraphRAG.



Garbage In = Garbage Out (GIGO - Dữ liệu rác tạo ra kết quả rác).



NER (Named Entity Recognition)

Nhận diện "Ai", "Cái gì", "Ở đâu" (Dùng spaCy, hoặc Prompting LLM).



Relation Extraction (RE)

Tìm ra cách các thực thể tương tác với nhau.



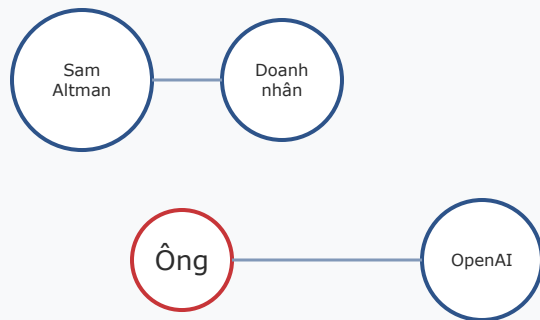
Thách thức

LLM mạnh nhưng chậm và đắt khi xử lý hàng ngàn trang tài liệu; NLP truyền thống nhanh nhưng kém linh hoạt.

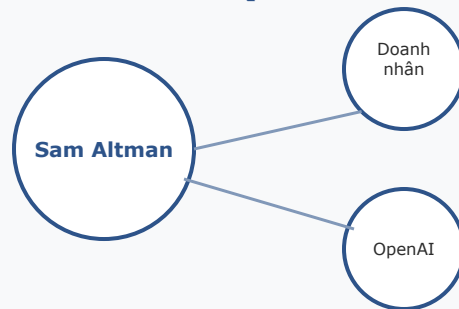
🧠 Văn bản: "Sam Altman là một doanh nhân. **Ông** đã thành lập OpenAI."
Nếu không quy chiếu "**Ông**" → "**Sam Altman**", đồ thị sẽ tạo ra một node bị cô lập tên là "Ông".

Bỏ qua bước này làm mất 30-40% các mối quan hệ quan trọng trong đồ thị.

Trước (Rời rạc)



Sau (Đã kết nối)





Ngôn ngữ có tính mơ hồ

"Apple báo cáo doanh số iPhone kỷ lục."

"Apple (quả táo) là một loại trái cây ngon."



Disambiguation đảm bảo hệ thống tạo ra hai node riêng biệt dựa trên ngữ cảnh lúc trích xuất.

Apple_Inc

Apple_Fruit



Nhiều đoạn văn bản có thể gọi cùng một thực thể theo các cách khác nhau.
VD: "**OpenAI**", "**Open AI**", "**open-ai**", "**OAI**".



Giải pháp: Thêm bước chuẩn hóa (normalization)

Sử dụng các kỹ thuật sau để gộp chúng thành một node duy nhất:

- Đối sánh chuỗi (String Matching)
- Vector similarity
- Sử dụng LLM



1. Đọc các đoạn văn bản (Chunks)

Phân tích và nạp dữ liệu thô từ nguồn tài liệu.



2. Giải quyết Đồng tham chiếu

Xử lý các đại từ (He/She/It) để xác định đúng thực thể.



3. Trích xuất Thực thể & Phân giải

Nhận diện và chuẩn hóa các đối tượng trong văn bản.



4. Trích xuất Mối quan hệ (Triples)

Xây dựng các liên kết Chủ thể - Quan hệ - Đối tượng.



5. Xóa trùng lặp và đẩy vào Neo4j

Tối ưu hóa dữ liệu và lưu trữ vào cơ sở dữ liệu đồ thị.

	NetworkX	Neo4j
Setup	pip install	Docker/Cloud
Scale	~100K nodes	Millions+
ACID	Không	Có
Algorithms	Có (BFS, PageRank)	Full library
Best for	Prototype	Production

Thêm **embeddings** cho mỗi node + source chunk references → enable hybrid search (graph + vector)

- **Bulk insert:** batch thay vì từng triple — 10× faster
- Beyond 100K: switch Neo4j hoặc **FalkorDB** (Redis-based)

Pipeline GraphRAG Tiêu chuẩn

Hành trình từ Câu hỏi của User đến Câu trả lời của LLM

3



-  **Query Processing:** Trích xuất thực thể từ câu hỏi.
-  **Seed Node Matching:** Tìm các thực thể đó (node gốc) trong Graph DB.
-  **Graph Traversal:** Khám phá khu vực xung quanh các node gốc.
-  **Textualization:** Chuyển đổi dữ liệu đồ thị trở lại thành văn bản.
-  **Generation:** LLM tổng hợp câu trả lời cuối cùng.

Câu hỏi

"Ai đồng sáng lập Microsoft?"

Trích xuất

LLM lấy ra các thực thể quan trọng:

[Microsoft]

Seeding (Khởi tạo)

Hệ thống tìm trong Graph DB node khớp hoàn toàn hoặc có ngữ nghĩa tương đồng với **"Microsoft"**.

→ Đây là điểm xuất phát cho quá trình duyệt đồ thị.

Thuật toán duyệt

Từ seed node, hệ thống dùng thuật toán Tìm kiếm theo chiều rộng (BFS) để lấy các nodes và edges lân cận.

Kết quả thu thập (Triples)

Nó thu thập tất cả Triples nối với node gốc:

- **(Bill Gates)**—[CO_FOUNDED]→**(Microsoft)**
- **(Paul Allen)**—[CO_FOUNDED]→**(Microsoft)**

1 Depth = 1: Quá nông

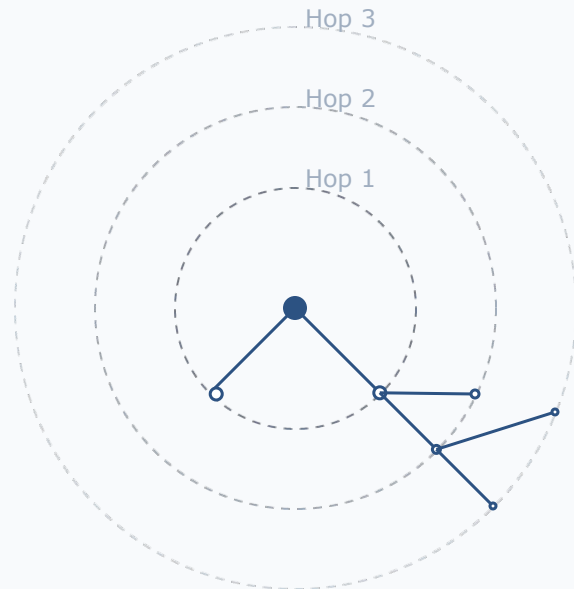
Chỉ lấy hàng xóm trực tiếp, dễ trượt mất ngữ cảnh "multi-hop".

★ Depth = 2: Tiêu chuẩn

Khuyến dùng. Lấy hàng xóm và ngữ cảnh lân cận của chúng.

⚠ Depth = 3+: Quá nhiều

Kéo theo dữ liệu không liên quan, làm quá tải Context Window của LLM và gây ra ảo giác.



Mô phỏng ranh giới truy vấn



Mục đích

LLM không thể "đọc" trực tiếp cơ sở dữ liệu đồ thị. Chúng ta phải chuyển subgraph thu được thành một prompt văn bản.



Quá trình chuyển đổi

RAW TRIPLES:

```
(Bill Gates, CO_FOUNDED, Microsoft), (Paul Allen, CO_FOUNDED, Microsoft)
```



VĂN BẢN HÓA:

"Các mối quan hệ sau tồn tại trong cơ sở tri thức: Bill Gates CO_FOUNDED Microsoft. Paul Allen CO_FOUNDED Microsoft."



Tích hợp Context

Đoạn văn bản subgraph được thêm vào prompt của LLM để định hướng kiến thức.



System Prompt

"Hãy trả lời câu hỏi của người dùng CHỈ sử dụng ngữ cảnh mối quan hệ được cung cấp sau đây."



Kết quả

Câu trả lời cực kỳ chính xác, bám sát thực tế và miễn nhiễm với ảo giác RAG thông thường.



Bổ trợ thay vì thay thế

GraphRAG không nên thay thế Vector Search; nó nên đóng vai trò bổ trợ lẫn nhau.



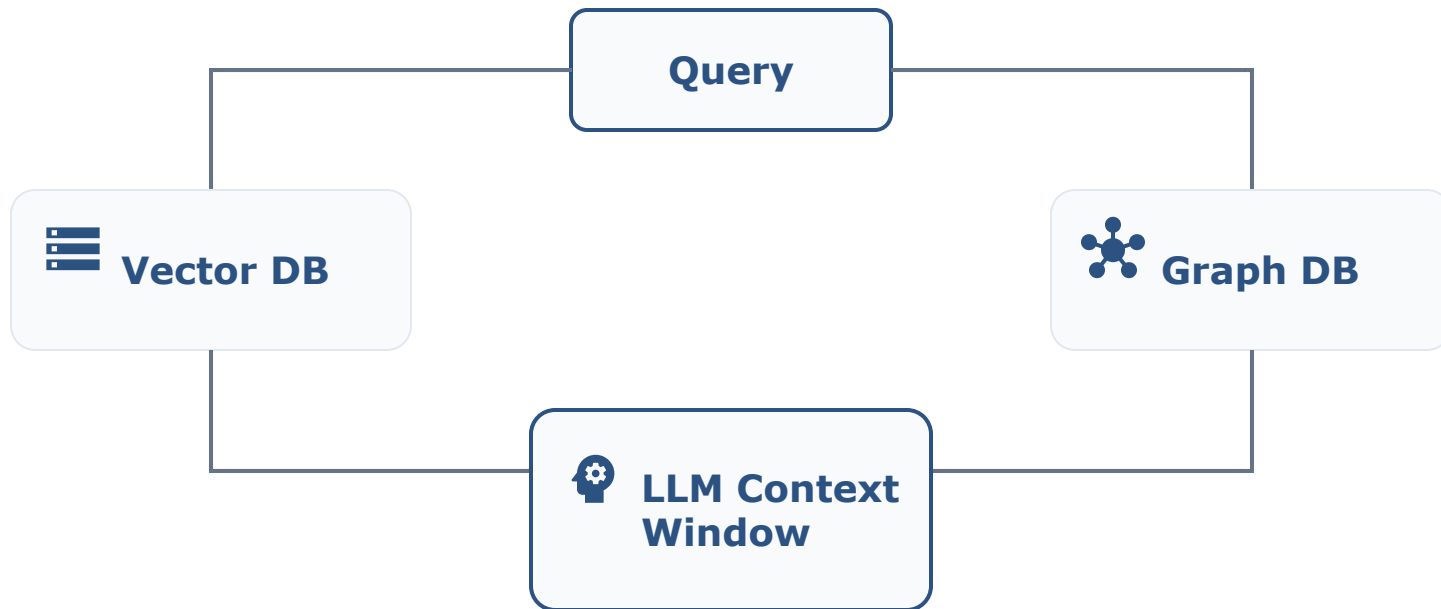
Hybrid Search

Chúng ta lưu trữ vector embeddings bên trong các node của đồ thị.



Cơ chế hoạt động

Dùng Semantic Search để tìm seed node, sau đó dùng Graph Traversal để tìm quan hệ, kết hợp cả ngữ cảnh cấu trúc lẫn phi cấu trúc.



- ✓ **Vector DB:** Tìm kiếm ngữ nghĩa để truy xuất các văn bản (chunks) liên quan.
- ✓ **Graph DB:** Duyệt đồ thị để khai thác các mối quan hệ thực thể phức tạp.



Đồ thị thông thường là tĩnh. Nhưng thế giới thực thay đổi.
"Ai là CEO của OpenAI?" (Đáp án năm 2018 khác 2024)



Giải pháp: Gắn mốc thời gian (validity dates) vào thuộc tính của các cạnh (edges).

Ví dụ minh họa:

Sam Altman

CEO_OF
{start: 2019, end: present}

OpenAI



Super-nodes (Node quá tải)

Một node như "USA" có thể có 100,000 kết nối.

Duyệt nó sẽ làm sập context window.



Đồ thị rời rạc

Trích xuất thất bại không nối được các subgraph.

Tạo ra các "hòn đảo" dữ liệu (Disconnected Components).



Ngữ cảnh nghèo nàn

Bước textualization lược bỏ quá nhiều sắc thái từ tài liệu gốc.

-  Luôn duy trì một con trỏ (pointer) từ Graph Node ngược về Document Chunk gốc.
-  Đặt giới hạn duyệt (VD: "duyệt 2 hop, nhưng tối đa 50 cạnh").
-  Liên tục tinh chỉnh prompt NER.

Các Kiến trúc SOTA (State-of-the-Art)

Các ông lớn công nghệ đang scale GraphRAG như thế nào.

4



GraphRAG cơ bản dựa trên trích xuất Triple đơn giản.



Các hệ thống SOTA hiện đại (2024+)

Tập trung vào **hiểu biết phân cấp (hierarchical understanding)** và giảm thiểu chi phí tính toán khổng lồ khi xây dựng đồ thị quy mô lớn.



Phát hành giữa năm 2024



Thiết kế cho Sensemaking

Tạo dựng ý nghĩa trên toàn bộ tập dữ liệu.



Triết lý cốt lõi

Không chỉ để trả lời câu hỏi cụ thể, mà để hỏi hệ thống:

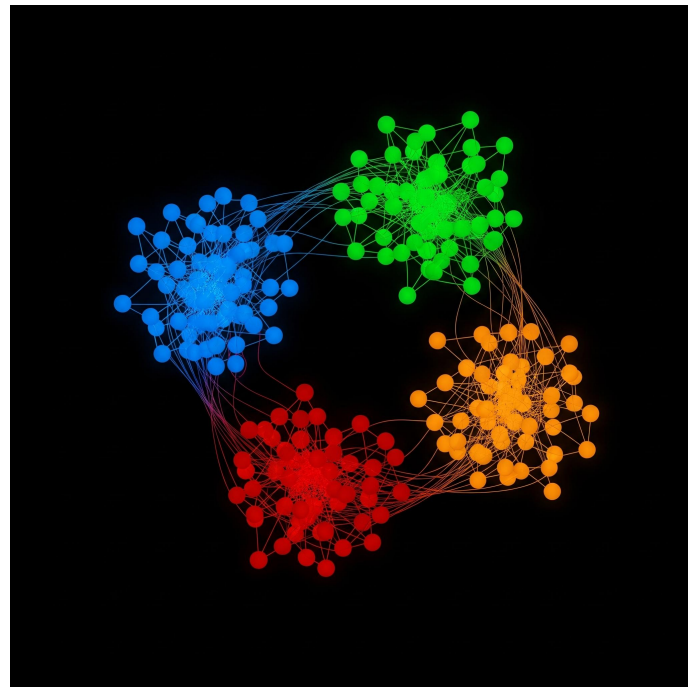
"Tập dữ liệu này nói về bức tranh tổng thể gì?"



Sử dụng **Thuật toán Leiden** để phát hiện các "Cộng đồng" (các cụm node liên kết chặt chẽ).



Gom nhóm thực thể thành "khu phố", tạo bản đồ phân cấp (từ vĩ mô đến vi mô).





Khi các cộng đồng hình thành, MS GraphRAG dùng LLM để tạo ra một "**Bản báo cáo tóm tắt**" cho từng cộng đồng ở mọi cấp độ.



Hệ thống tính toán trước (pre-compute) câu trả lời cho các câu hỏi mang tính chủ đề toàn cục.



Local Search

Cho câu hỏi cụ thể ("Ai kết nối X với Y?"). Duyệt qua các nodes.



Global Search

Cho câu hỏi chủ đề ("Đâu là rủi ro chính trong các hợp đồng này?"). Đọc các Báo cáo Cộng đồng đã tạo sẵn thay vì duyệt node thô.



- **Chi phí cực đắt:** Việc tạo tóm tắt cho mọi cộng đồng trên kho dữ liệu lớn tốn một lượng token khổng lồ trong quá trình indexing.
- Index 1 triệu token có thể tốn \$10–\$50+ chỉ để xây đồ thị.



Không phù hợp cho dữ liệu thay đổi liên tục.



Tối ưu hiệu năng

Giải quyết vấn đề chi phí và sự chồng chéo của MS GraphRAG.



Cải tiến cốt lõi: Truy xuất hai cấp độ (Dual-level retrieval)

Trích xuất cả thực thể chi tiết lẫn khái niệm trừu tượng mà không cần tạo trước các báo cáo cộng đồng đặt đồ.



Cấu trúc Embedding đa diện

Thay vì chỉ embed các chunks, LightRAG tạo vector embedding cho cả **Nodes** và **Edges**.



Truy xuất cực nhanh

Dùng vector search để tìm ra nodes VÀ relationships liên quan cùng lúc, bỏ qua việc tính toán duyệt đồ thị nặng nề.

Bảng So sánh (MS GraphRAG vs. LightRAG vs. Flat RAG)

	Chi phí Index	Khả năng Global	Multi-hop
Flat RAG	 Rất Rẻ	 Kém	 Kém
MS GraphRAG	 Rất Đắt	 Xuất sắc	 Xuất sắc
LightRAG	 Trung bình	 Tốt	 Xuất sắc (Triển khai nhanh)



Đo lường RAG truyền thống đã khó, đo lường GraphRAG còn khó hơn vì bạn không chỉ đánh giá khả năng trích xuất văn bản mà còn phải đánh giá khả năng suy luận trên cấu trúc đồ thị.



Tính toàn diện

Câu trả lời có giải quyết trọn vẹn và đầy đủ câu hỏi không?



Tính đa dạng

Thông tin cung cấp có phong phú và đa chiều không?



Tính chính xác

Mức độ tin cậy của các sự kiện được nêu ra.



Phải benchmark trên các tập dữ liệu thiết kế riêng cho suy luận multi-hop (VD: HotpotQA, 2WikiMultihopQA).



GraphRAG luôn cho thấy độ chính xác cao gấp **2-3 lần** so với Flat RAG trên các tập này.

Chiến lược Doanh nghiệp & ROI

Cách chứng minh giá trị của GraphRAG với
stakeholder

5



Flat RAG

- Rẻ khi xây dựng
- Rẻ khi truy vấn



GraphRAG

- **Rất đắt** khi xây dựng (vì LLM trích xuất triples)
- Truy vấn **khá rẻ** và **cực kỳ chính xác**



Rule of thumb

Đừng dùng GraphRAG cho các tài liệu dùng một lần. Hãy dùng nó cho tri thức cốt lõi của doanh nghiệp.



Flat RAG Optimization

- Tra cứu tài liệu IT Support (Factoid lookup).
- Chatbot chăm sóc khách hàng cơ bản.
- Tra cứu quy định nhân sự đơn lẻ.



Nếu đáp án nằm rõ ràng trong 1 đoạn văn, hãy dùng Flat RAG.



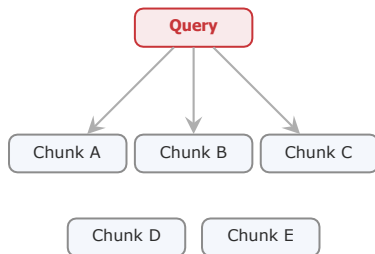
GraphRAG Upgrade Criteria

- Điều tra báo chí / Phân tích tình báo.
- Đánh giá tài liệu y khoa chuyên sâu.
- Thẩm định pháp lý (Legal discovery) phức tạp.



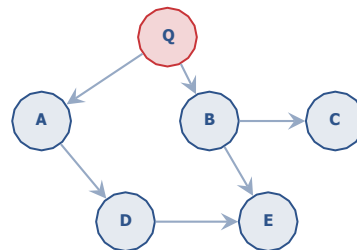
Nếu câu trả lời cần việc tổng hợp thông tin từ nhiều tài liệu khác nhau, hãy dùng GraphRAG.

Vector RAG



Top-K similarity

GraphRAG



Graph traversal (2 hops)

Single-doc factoid → **Flat RAG**. Multi-entity relations → **GraphRAG**.
Thematic overview → **GraphRAG global**. **Hybrid**: detect query type rồi route tương ứng.

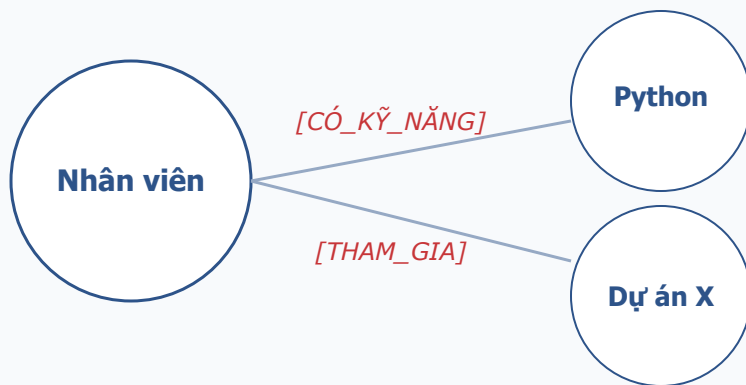
Lập bản đồ phụ thuộc hợp đồng

-  Cho phép luật sư thấy ngay lập tức sự thay đổi của một quy định vĩ mô sẽ tác động đến từng hợp đồng vendor nhỏ lẻ như thế nào.



Tạo ra "Bộ não" nội bộ cho công ty

-  Kết nối tri thức phân tán để tìm đúng người, đúng việc thông qua các mối quan hệ thực tế trong dự án và kỹ năng.

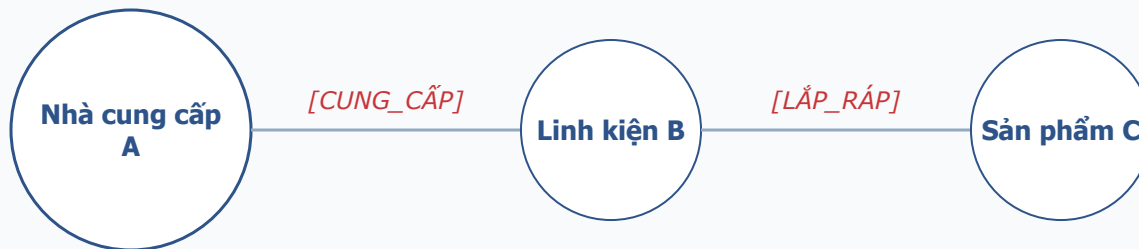


Câu hỏi truy vấn:

"Tìm cho tôi người biết Python và từng làm một dự án tương tự Dự án X."

Ảnh xạ thể giới vật lý

- Graph Traversal giúp xác định ngay lập tức các điểm nghẽn và tác động dây chuyền khi một mắt xích trong chuỗi cung ứng gặp sự cố.



Câu hỏi:

"Nếu Nhà cung cấp A ngừng hoạt động, sản phẩm nào bị trễ?"

Demo & Thực hành

6

1. Corpus: 100 Wikipedia articles AI companies, extract entities → Neo4j graph
2. Query 1 (flat RAG OK): “What is OpenAI?” — cả hai pipeline đúng
3. Query 2 (multi-hop): “AI companies co-founded by former Google employees” — flat RAG hallucinate, GraphRAG đúng
4. Visualization: Neo4j Browser hiển thị subgraph, highlight answer nodes

Mục tiêu: Build Knowledge Graph + GraphRAG agent, multi-hop accuracy vượt flat RAG + 20%

Deliverable: Knowledge graph visualization + GraphRAG benchmark report (multi-hop accuracy, latency, cost)

Thời gian: 2 giờ

1. **Entity extraction:** Dùng LLM-based NER trên domain corpus, output triples (subject, predicate, object)
2. **Build graph:** Load triples vào NetworkX (hoặc Neo4j), thêm embeddings cho nodes
3. **GraphRAG retrieval:** Implement query → seed nodes → BFS traversal → subgraph-to-text → LLM generate
4. **Benchmark:** So sánh GraphRAG vs Flat RAG trên 20 multi-hop questions — đo accuracy, latency, cost

GitHub repo + Neo4j/NetworkX visualization + benchmark report: bảng accuracy, phân tích failure modes

Những ý chính cần nhớ sau buổi học hôm nay

- 1 Knowledge graphs enable multi-hop reasoning mà flat RAG không làm được — dùng cho relational queries
- 2 Entity extraction quality là bottleneck — invest NER + coreference resolution trước khi build graph
- 3 “Graph quality beats Graph size” — 1000 high-quality triples beats 100K noisy ones
- 4 GraphRAG pipeline là production-ready starting point — customize entity extraction cho domain của bạn

Ngày 20: Multi-Agent Systems

"Single agent mạnh, tiếp theo scale lên Multi-Agent — supervi-sor, debate, parallel"

- Hoàn thành Lab 19: GraphRAG agent + benchmark report
- Đọc: Anthropic "Building Effective Agents" (2024)

Hỏi & Đáp

Khi nào nên dùng GraphRAG thay Flat RAG? Hybrid approach có đáng đầu tư không?



Cảm ơn!

AICB-P2T3 · Ngày 19 · GraphRAG & Knowledge Graphs

`github.com/vinuni-aicb`

Liên hệ: instructor@vinuni.edu.vn